

류기현 Frontend Developer

React 기반의 의료 제품과 사내 시스템을 개발하고 있습니다. 심전도 그래프의 태블릿 인터랙션을 설계하고, 사내 디자인 시스템 3.0 개편을 주도했습니다.

제품을 만들면서 반복해서 생기는 문제를 공통 구조나 자동화 도구로 해결하는 데 관심이 있습니다. 최근에는 여러 저장소의 코드 변경 영향을 AI로 분석해 리뷰 맥락을 제공하는 도구도 만들었습니다.

khryu0610@gmail.com [GitHub](#)[↗] [Blog](#)[↗]

경력

메디컬에이아이 서울

2024.11 - 현재

소프트웨어그룹 프로덕트본부 FE팀 · Frontend Developer

대표 프로젝트

MAIUI 3.0 메디컬에이아이

2026.03 ~ 현재

제품에서 안전하게 확장·조합할 수 있는 컴포넌트 구조와 디자인 자산 자동화 체계를 만든 사내 디자인 시스템 개편

핵심 기여

제품에서 확장 가능한 컴포넌트 구조로 개편

- 기존 디자인 시스템은 컴포넌트마다 API와 스타일링 방식이 달랐습니다. 제품에서 필요한 스타일을 적용하려고 내부 DOM을 직접 선택해 덮어쓰는 경우도 있었고, 작은 요구사항에도 디자인 시스템을 수정하고 다시 배포해야 했습니다.
- 제품별 요구사항을 디자인 시스템에 계속 추가하는 방식보다, 공통 컴포넌트는 책임을 명확히 하고 제품에서 필요한 부분은 안전하게 조합할 수 있게 만드는 것이 낫다고 판단했습니다. 컴포넌트 API와 스타일링 컨벤션을 다시 정의하고, 기존의 간편한 사용 방식과 조합 가능한 형태를 함께 제공했습니다.
- 제품 코드가 디자인 시스템의 내부 구현에 직접 의존하는 경우를 줄이고, 컴포넌트를 수정하지 않고도 제품에서 필요한 UI를 조합할 수 있는 방향으로 구조를 바꿨습니다.

디자인 자산 갱신을 자동화

- 아이콘과 디자인 토큰의 변경 사항을 사람이 확인하고 코드로 옮기고 있었습니다. 기존 아이콘 변경 이력을 분석해보니 갱신 간격은 평균 90일이었고, 임의 시점의 예상 대기 기간은 64일이었습니다. 개편 당시에는 Figma의 아이콘 376개 중 245개만 코드에 반영돼 약 35%가 누락된 상태였습니다.
- 누락분을 한 번 정리하는 것보다 같은 문제가 다시 쌓이지 않게 만드는 것이 중요하다고 봤습니다. 디자인 자산을 코드와 전체 동기화하고, 이후 변경 감지부터 코드 생성, 변경 내역 작성, PR 생성까지 이어지는 파이프라인을 구축했습니다.
- 아이콘 반영률을 65%에서 100%로 높였고, 분기 단위로 발생하던 반영 대기를 수일 수준으로 줄였습니다. 구축에는 12작업일이 들었고, 반복 작업 절감까지 포함하면 1년 이내에 구축 비용을 회수할 수 있는 수준이었습니다.

여러 제품의 컴포넌트 사용 맥락을 AI로 분석해 코드 리뷰와 디자인 QA에 필요한 변경 영향 정보를 제공한 도구

핵심 기여

정적 검색으로 찾기 어려운 변경 영향을 AI로 분석

- ❗ 공통 모달 개편을 앞두고 10개 제품에서 영향을 받는 화면을 확인해야 했습니다. 단순 사용처는 코드 검색으로 찾을 수 있었지만, 실제 진입 경로와 노출 조건, 제품별 수정 정도, 변경 위험도는 코드를 직접 읽어야 판단할 수 있었습니다.
- ✅ 사람이 리뷰할 때 확인하던 내용을 15개 항목으로 나누고, AI가 저장소의 코드 맥락을 읽어 같은 형식으로 결과를 작성하도록 했습니다. 사용 위치, 진입 경로, 노출 조건처럼 코드 맥락이 필요한 부분은 AI가 말고, 결과 형식과 허용 값, 중복 여부처럼 규칙으로 판단할 수 있는 부분은 코드로 다시 검증했습니다.
- ✔ 3일 동안 4개 제품에서 132건의 모달 사용 사례를 확보했고, 이 중 36건인 약 27%는 서버 오류, 시간 초과, 유효성 검사 실패처럼 정상적인 화면 흐름만 따라가서는 놓치기 쉬운 사용처였습니다.

분석 결과를 리뷰에서 바로 쓸 수 있게 제공

- ❗ 사용처를 찾아도 코드 위치만 나열하면 리뷰어가 다시 제품을 열어 진입 경로와 위험도를 확인해야 했습니다.
- ✅ 저장소별 결과를 자동으로 모으고 유형, 위험도, 확인 여부에 따라 탐색할 수 있는 대시보드를 만들었습니다. AI가 판단한 내용에는 근거와 확신도를 함께 남기고, 사람의 최종 확인 결과와 구분했습니다.
- ✔ 코드 리뷰에서 변경 코드와 함께 제품별 사용 맥락을 확인할 수 있게 했고, 같은 자료를 디자인 QA에서도 사용해 개발과 디자인이 동일한 검토 범위를 공유하도록 했습니다.

태블릿에서 확대·이동·정밀 측정이 서로 방해하지 않도록 터치 입력을 판별하고 제어한 심전도 그래프 측정 도구

핵심 기여

한 가지 터치 입력에서 여러 동작을 구분

- ❗ PC에서는 마우스 버튼과 키보드로 확대, 이동, 측정을 구분할 수 있었지만 태블릿에서는 모두 손가락 터치로 시작했습니다. 실제로 핀치 확대 중 측정점이 생기거나 그래프를 이동하려다 측정이 시작되는 문제가 있었습니다.
- ✅ 터치가 시작된 순간 바로 기능을 정하지 않고, 유지 시간과 이동 거리, 손가락 수를 보고 사용자의 의도를 판단하도록 했습니다. 한 동작이 시작된 뒤에는 다른 기능이 같은 입력을 동시에 처리하지 않도록 공통 제어 상태를 두었습니다.
- ❌ 확대, 이동, 측정, 정밀측정, 도구 드래그가 같은 화면 위에서 동작하더라도 서로 입력을 빼앗거나 동시에 실행되는 문제를 줄였습니다.

일시적인 인터랙션이 기존 상태를 깨지 않게 설계

- ❗ 측정 도구를 드래그하는 동안 확대나 측정 기능을 잠시 막아야 했지만, 드래그가 끝난 뒤 사용자가 원래 사용하던 상태로 자연스럽게 돌아가야 했습니다.
- ✅ 드래그 진입 시 현재 상태를 저장하고, 동작 중에는 필요한 기능만 점유한 뒤 종료 시 이전 상태를 복원하도록 했습니다. 드래그 자체의 동작과 태블릿에서 필요한 상태 제어도 분리해 데스크톱과 공통으로 사용할 수 있게 했습니다.
- ❌ 일시적인 도구 조작 때문에 사용자가 선택해둔 측정 상태가 사라지거나 다른 인터랙션이 끼어드는 문제를 막았습니다.

브라우저에서 실제 입력 충돌까지 검증

- ❗ 심전도 그래프에서는 화면 좌표가 실제 시간과 전압 값으로 변환되기 때문에, UI가 자연스럽게 움직이는 것뿐 아니라 측정 좌표가 정확해야 했습니다. 핀치와 스크롤 간섭처럼 실제 브라우저 런타임에서만 확인 가능한 문제도 있었습니다.
- ✅ 좌표 변환과 상태 로직은 단위 테스트로 분리하고, 입력에 따른 상태 전이와 측정점 생성·제거, UI 변화는 브라우저 통합 테스트로 검증했습니다. 모바일 디바이스 에뮬레이션에서는 핀치 줌과 측정 충돌, 스크롤 간섭도 함께 확인했습니다.
- ❌ 순수 로직뿐 아니라 실제 터치 입력부터 화면 변화까지 이어지는 동작을 테스트 범위에 포함해 태블릿 환경의 회귀 가능성을 줄였습니다.

회사의 자산, 매출, 거래처를 관리하는 시스템을 개발하고 대용량 파일 다운로드 구조를 개선

핵심 기여

대용량 파일 다운로드를 스트림 기반으로 변경

- ❗ 서버가 파일 전체를 압축한 뒤 응답하고 브라우저가 다시 후처리하는 구조에서 파일이 커질수록 요청이 실패했습니다. 1GB 파일을 처리할 때는 브라우저 메모리 사용량이 약 1.9GB까지 증가했습니다.
- ✅ 처음에는 네트워크 문제를 의심했지만 백엔드와 처리 과정을 확인하면서, 전체 파일이 준비될 때까지 기다리고 다시 메모리에 쌓는 구조 자체가 병목이라고 판단했습니다. 서버는 데이터를 생성되는 대로 보내고, 클라이언트도 받은 데이터를 메모리에 계속 쌓지 않고 가능한 환경에서는 바로 파일로 기록하도록 변경했습니다.
- 🔴 100MB부터 1GB까지 파일 크기와 동시 요청 수를 달리해 검증했으며, 기존 방식에서 실패하던 대용량 다운로드를 모두 완료할 수 있었습니다. 1GB 처리 시 브라우저 메모리 사용량도 수십 MB 수준으로 낮았습니다.

보안 원칙을 유지하면서 클라이언트 후처리

- ❗ 다운로드 파일을 암호화해야 했지만, 비밀번호를 서버에 전달하지 않는 기존 보안 원칙을 유지해야 했습니다.
- ✅ 암호화에 필요한 후처리는 클라이언트에 남기되, 이 과정에서도 파일 전체를 한 번에 메모리에 올리지 않도록 스트림 단위로 처리했습니다.
- 🔴 비밀번호를 서버로 전달하지 않으면서도 대용량 파일 다운로드를 안정적으로 처리할 수 있는 구조를 만들었습니다.

스크럼 운영

- ❗ 여러 기능이 병렬로 개발되면서 작업 현황과 일정, 블로커를 팀 단위로 계속 맞출 필요가 있었습니다.
- ✅ 주간 회의와 스프린트 회고를 운영하고 작업 현황, 이슈, 예상 일정을 정리해 우선순위를 조율했습니다.
- 🔴 개발 과정에서 막힌 지점과 일정 변화를 팀이 지속적으로 공유할 수 있는 운영 흐름을 만들었습니다.

개인 프로젝트

실시간 의결 투표 플랫폼 [GitHub](#)

2인

의사정족수와 의결정족수를 지원하는 실시간 투표 시스템

핵심 기여

SSE와 REST 사이의 데이터 덮어쓰기 방지

- ❗ SSE 이벤트와 REST 응답이 함께 들어오는 상황에서 늦게 도착한 오래된 이벤트가 더 최신인 서버 응답을 덮어쓰는 문제가 있었습니다.
- ✅ 이벤트를 받은 시점과 실제 캐시에 반영하는 시점에 각각 시간을 비교해, 현재 상태보다 오래된 이벤트는 반영하지 않도록 했습니다.
- 🔴 SSE와 REST 응답 순서가 뒤바뀌어도 오래된 데이터가 최신 상태를 덮어쓰지 않도록 처리했습니다.

SSE 재연결 시 유실 구간 복구

- ❗ 브라우저 탭 비활성화 등으로 SSE 연결이 끊기면 그 사이 서버에서 발생한 이벤트를 놓칠 수 있었습니다.
- ✅ 최초 연결과 재연결을 구분하고, 실제 연결 중단이 발생한 경우에만 서버 데이터를 다시 조회하도록 했습니다.
- 🔴 연결 복구 시 불필요한 재요청은 줄이면서 끊긴 구간의 데이터를 다시 동기화할 수 있게 했습니다.

카카오맵 API를 Vue 3 컴포넌트로 사용할 수 있게 만든 오픈소스 라이브러리

핵심 기여

외부 프로젝트에서 실제로 설치 가능한 패키지 구조 구축

- ❗ 초기 버전은 npm 설치 과정에서 실패하거나 배포 파일의 진입점을 제대로 찾지 못하는 문제가 있었고, 저장소 설정과 개발용 파일도 함께 배포되고 있었습니다.
- ✅ Vite 기반 라이브러리 빌드로 구조를 정리하고 ESM과 CommonJS 진입점, 타입 선언 생성을 나눠 소비자 프로젝트가 필요한 파일만 가져갈 수 있도록 패키지 구성을 다시 잡았습니다.
- ✅ 내부 개발 환경에서만 동작하던 형태를 실제 외부 프로젝트에서 설치하고 사용할 수 있는 npm 패키지로 정리했습니다.

모듈 해석 환경별 타입 문제 개선

- ❗ 일반적인 Vue 프로젝트에서는 문제없이 동작했지만 일부 Node·CommonJS 환경에서는 타입이나 서브패스를 찾지 못했고, CJS 산출물이라고 생각했던 파일이 실제로는 예상과 다른 형식으로 만들어지는 경우도 있었습니다.
- ✅ 코드와 타입 진입점을 환경별로 다시 나누고, 타입 선언 내부 경로가 소비자 설정에 따라 깨지지 않도록 생성 방식을 변경했습니다. 같은 문제가 반복되지 않도록 빌드 산출물 검증도 자동화했습니다.
- ✅ 주요 모듈 해석 조건에서 발생하던 타입 오류를 제거하고, 여러 소비자 환경에서 패키지를 일관되게 해석할 수 있도록 만들었습니다.

실제 사용자가 있는 오픈소스 운영

- ❗ 라이브러리를 공개한 뒤에는 개발팀이 예상하지 못한 사용 방식과 문서 오류가 실제 사용자로부터 들어오기 시작했습니다.
- ✅ npm 메인테이너로 릴리스를 맡고 오픈채팅의 사용자 피드백과 외부 사용자의 문서 PR을 직접 검토해 반영했습니다.
- ✅ 프로젝트를 일회성 과제로 끝내지 않고 외부 사용자의 피드백을 받아 개선하는 오픈소스로 운영했으며, SSAFY 자율 프로젝트 우수팀으로 선정됐습니다.

학력

연세대학교
문과대학 중어중문학과
2018.03 - 2023.08

서울

교육

SSAFY 삼성청년소프트웨어 아카데미
비전공 Java반
2023.07 - 2024.06

서울

멋쟁이사자처럼 웹 프론트엔드 스쿨
프론트엔드 중점 과정
2022.07 - 2023.01

온라인

자격 및 언어

자격증 **SQLD**
언어 **English (OPIc IM2)**